

Frontend Knowledge Base & API Contract Reference

A complete technical reference for the GrowVio Frontend repository — architecture review, domain model, and the full backend API surface consumed by the User App and Admin Portal, with sample request/response contracts for every endpoint.

Repository: GrowVio_FrontEnd (frontend + admin)

Prepared: 11 August 2026

Scope: Frontend applications only — backend is a separate repository (GrowVio_Backend)

Apps covered: User App (React 18, Vite, port 3000) · Admin Portal (React 19, Vite, port 5174)

Table of Contents

1	Executive Summary	6	API Gateway Routing Map
2	Technology Stack	7	Complete API Inventory
3	Repository & Folder Structure	7.1	Authentication & Session
4	Architecture Review	7.2	User Profile, Notes & History
4.1	Application Topology	7.3	Books — Public/User
4.2	Routing & Code-Splitting	7.4	Payments
4.3	State Management	7.5	Gamification
4.4	Authentication & Authorization	7.6	Social / Reviews
4.5	API Client Layer	7.7	Notifications
4.6	Component & UI Architecture	7.8	AI Generation (Backend)
4.7	Environment Configuration	7.9	Admin — Books CRUD & Publish
4.8	Build, Quality & Testing	7.10	Admin — Content / Levels Editor
4.9	Observability & 3rd-Party Integrations	7.11	Admin — File Uploads
4.10	Strengths	7.12	Admin — Users & Analytics
4.11	Risks & Recommendations	8	Environment Variables Reference
5	Domain Model	9	Route Map Appendix

1. Executive Summary

The **GrowVio_FrontEnd** repository contains two independently deployable single-page applications that together form the client side of GrowVio, a gamified book-summary platform targeted at Indian professionals aged 22–40:

Application	Path	Audience	Stack	Dev Port
User App ("frontend")	frontend/	End users (readers)	React 18 + TypeScript + Vite + Tailwind + shadcn/ui + Radix	3000
Admin Portal ("admin")	admin/	Content creators / admins	React 19 + TypeScript + Vite + Tailwind + Tiptap	5174

Both apps are pure clients — this repository holds **no backend code**. All data comes from a Spring Cloud Gateway (`api-gateway` , port 8080) that fronts five microservices: **user-service**, **content-service** (aka book-service), **ai-service** (Python), **social-service**, **gamification-service** and **notification-service**, plus direct browser calls to **Razorpay** (payments), **Sentry** (error tracking), **Google Analytics**, and — from the Admin Portal and a few internal tool pages — direct calls to the **Anthropic Claude** and **Groq** LLM APIs.

The product's core mechanic is a **7-level "Knowledge Pyramid"** per book (Snapshot → Flashdeck → Infosummary → Deep Read → Mastery Quiz → Action Plan → Revision Booklet), gated by a 4-tier subscription plan (FREE / STARTER / PREMIUM / PRO) and monetized via Razorpay.

This document reviews the frontend architecture as it stands today, documents the full domain model, and catalogs every backend/external API endpoint the two apps call — **~47 backend REST endpoints** across 6 microservices, plus 3 direct third-party integrations — each with a sample request/response contract derived directly from the TypeScript service layer.

2. Technology Stack

Layer	User App	Admin Portal
Framework	React 18.3	React 19.2
Language	TypeScript 5.8	TypeScript 6.0 (beta channel)
Build tool	Vite 5.4 + SWC plugin	Vite 8.0
Routing	react-router-dom v6	react-router-dom v7
Server state	@tanstack/react-query v5	@tanstack/react-query v5
Styling	Tailwind CSS 3.4 + tailwindcss-animate + class-variance-authority	Tailwind CSS 4.2 (CSS-first config)
Component primitives	Radix UI (~25 packages) wrapped as shadcn/ui in <code>components/ui</code>	Custom components, no shadcn layer
Forms	react-hook-form + zod v3 + @hookform/resolvers	react-hook-form + zod v4 + @hookform/resolvers

Rich text editing	—	Tiptap v3 (starter-kit, image, link, highlight) + @uiw/react-md-editor
HTTP client	axios (singleton instance with interceptors)	axios (singleton instance with interceptors)
Charts	recharts	—
Animation	framer-motion	—
Error tracking	@sentry/react	@sentry/react
Unit / component tests	Vitest + @testing-library/react + jsdom	—
E2E tests	@playwright/test (in <code>frontend/tests/e2e</code> , covers both apps)	—
LLM SDKs used directly from browser	Groq REST API (internal tool pages only)	@anthropic-ai/sdk (Claude, <code>dangerouslyAllowBrowser: true</code>)
Payments	Razorpay Checkout.js (loaded via <code>index.html</code> script tag)	—

3. Repository & Folder Structure

```

GrowVio_FrontEnd/
├── frontend/                                     # User App (React 18)
│   ├── src/
│   │   ├── pages/                             # 40+ route-level page components
│   │   │   ├── admin/                         # ⚠ legacy in-app admin pages (see §4.11)
│   │   │   ├── settings/                     # profile / notifications / subscription / goals
│   │   │   └── tools/                         # internal team tools (WarRoom, TaskTracker, GrowVioAgents)
│   │   ├── components/
│   │   │   ├── ui/                           # shadcn/ui primitives (button, dialog, table, ...)
│   │   │   ├── landing/, books/, layout/, common/
│   │   ├── services/                         # ← axios API layer, one file per backend domain
│   │   ├── context/AuthContext.tsx           # global auth state (JWT + user)
│   │   ├── config/env.ts                     # typed import.meta.env wrapper
│   │   ├── constants/levelNames.ts           # the 7-level pyramid definition
│   │   ├── types/index.ts                    # shared domain types
│   │   └── App.tsx                           # router + providers root
│   └── tests/e2e/                             # Playwright suites (covers frontend + admin)
├── admin/                                     # Admin Portal (React 19)
│   └── src/
│       ├── pages/                             # Login, Dashboard, Books*, ContentEditor, RichEditor,
│       └── AbTest, ContentCosts
│           ├── components/editor/              # Tiptap-based rich content editor + AI draft panel
│           ├── services/                       # ← axios API layer
│           ├── hooks/                          # useAuth, useBooks, useContent (TanStack Query wrappers)
│           ├── context/AuthContext.tsx
│           └── types/book.types.ts
├── package.json                               # root – unrelated stray @anthropic-ai/sdk dep (see §4.11)
└── README.md

```

4. Architecture Review

Scope: this review covers the **frontend repository only**. Backend service internals are referenced only where needed to explain how the frontend integrates with them (see §6).

4.1 Application Topology

The two apps are fully independent Vite SPAs — separate `package.json`, separate `node_modules`, separate deploy targets — that both talk to the same API Gateway. They share no code (no shared package/workspace), which means duplicated concerns: both ship their own axios client, auth context, JWT-storage logic and `authService.login` call. This is a reasonable trade-off for two teams/apps with very different UX needs (consumer app vs. internal tool), but it does mean fixes to cross-cutting concerns (e.g. token-refresh behaviour) must be made twice.

4.2 Routing & Code-Splitting

- **User App:** a single flat `<Routes>` tree in `App.tsx` with ~45 routes. Auth-critical and small pages (landing, login, signup, book detail) are statically imported for fast first paint; all "heavy" authenticated pages (Dashboard, BookReading, Quiz, admin pages, internal tools) are `React.lazy`-loaded, each becoming its own chunk, wrapped in one shared `<Suspense>` with a spinner fallback. Route guarding is done with wrapper components (`ProtectedRoute` , `AdminRoute`) rather than a data-router loader pattern.
- **Admin Portal:** two-tier router — an outer `<Routes>` handles `/login` vs. everything else; everything else is wrapped once in `ProtectedRoute` + a persistent `Layout` (sidebar shell), inside which a second `<Routes>` handles the 9 authenticated pages. No lazy-loading is used here — acceptable given the smaller page count and internal-only audience.

4.3 State Management

- **Server state:** TanStack Query in both apps. The Admin Portal wraps every service call in a custom hook (`useBooks` , `useContent` , ...) with consistent query-key conventions and cache invalidation on mutation — a clean, idiomatic pattern. The User App calls services directly from page components via `useQuery` /inline `useEffect` + `useState` in places, which is less consistent (see §4.11).
- **Client/global state:** React Context for auth only (`AuthContext` in each app). No Redux/Zustand — appropriate for this app's size.
- **Persistence:** JWT and user object are persisted to `localStorage` directly (keys `token` / `user` in the User App, `admin_token` / `admin_user` in Admin) — see §4.11 for the XSS-exposure trade-off this implies.

4.4 Authentication & Authorization

- Both apps authenticate against the same backend endpoint, `POST /api/auth/login`, which returns a JWT plus the user record (id, name, email, role, plan) in one flat JSON body.
- **User App** models four roles (`USER` , `CONTENT_CREATOR` , `ADMIN` , `SUPER_ADMIN`) and derives boolean flags (`isAdmin` , `isPremium` , ...) from `role` / `plan` client-side; route guards (`ProtectedRoute` , `AdminRoute` , `SuperAdminRoute` , `ContentCreatorRoute`) branch on these. On mount, a stored token is validated by calling `GET /api/users/me`; failure clears storage and logs the user out silently.

- **Admin Portal** only recognises the literal string `'ADMIN'` in its `ProtectedRoute` — a `CONTENT_CREATOR` or `SUPER_ADMIN` user who successfully logs in is shown a hard "Access Denied" screen rather than a scoped view (see §4.11).
- All authorization is **client-side only** for UX purposes; the actual enforcement is the API Gateway's `JwtAuthenticationFilter` / `AdminRoleFilter` and each service's own checks — correctly, the frontend never trusts itself as the security boundary.

4.5 API Client Layer

- Every backend call in both apps goes through **exactly one** axios singleton per app (`services/apiClient.ts`), confirmed by a repo-wide search — no stray `fetch()` or ad-hoc axios calls to the gateway exist outside the services layer. This is a genuine architectural strength.
- **User App** interceptor is the more mature of the two: it stamps request start-time for duration logging, skips attaching a stale Bearer token on `/api/auth/login` and `/api/auth/signup`, structured-logs every request/response via a custom `logger`, and branches its 401/403/429/500/503 handling (401 → hard redirect to `/login` after clearing storage). Login specifically overrides the default 15s timeout to 120s to tolerate free-tier cold starts.
- **Admin Portal** interceptor is minimal: attach token if present, redirect to `/login` on 401. No request logging, no differentiated error handling for 403/500/503.
- Base URL resolution differs between apps: User App falls back silently to `http://localhost:8080` if `VITE_GATEWAY_URL` is unset; Admin Portal **throws at import time** if `VITE_API_URL` is missing — a stricter, fail-fast choice.

4.6 Component & UI Architecture

- User App follows the shadcn/ui convention: unstyled Radix primitives copied into `components/ui/` and styled with Tailwind + `class-variance-authority`, giving full control without a component-library dependency. Domain components are organised by feature folder (`books/`, `landing/`) and two shell layouts (`AppShell`, `AdminShell`) wrap page content.
- Admin Portal is hand-rolled (no shadcn layer) — components use inline style objects/hex colors in places (e.g. `ProtectedRoute`'s access-denied screen) rather than a design-token system.
- Admin's content editor (`components/editor/`) is the most sophisticated UI subsystem in the repo: Tiptap-based rich text editing with a custom PDF-embed extension, an AI-draft side panel, and a supporting-documents panel — built to support the multi-step "AI generates draft → human edits → publish" content workflow.

4.7 Environment Configuration

- Both apps use Vite's standard `.env.[mode]` / `.env.[mode].local` layering. The User App additionally defines `.env.dev`, `.env.staging`, `.env.preprod` — non-standard filenames that Vite only loads when the dev/build command is run with an explicit `--mode` flag (e.g. `vite --mode dev`); the plain `npm run dev` script does not pass one, so locally the app falls back to hard-coded defaults unless a `.env.local` is present. This is documented behaviour but easy to trip over (see §4.11).
- An `EnvBanner` component in the User App visibly flags non-production environments in the UI — a nice guard against accidentally treating a staging session as prod.

4.8 Build, Quality & Testing

- ESLint 9 (flat config) + typescript-eslint in both apps; `tsc -b` as a separate typecheck step before Vite build in the Admin Portal.
- User App has both unit tests (Vitest, currently a single example spec) and a substantial Playwright E2E suite (8 numbered suites covering auth, books, profile, navigation, payment, deep user-portal flows, the admin portal, and raw API coverage) with recorded HTML reports and screenshot sequences — a genuinely strong E2E investment for a project this size.
- Admin Portal has no test suite of its own; it is covered only indirectly by the User App's `test:e2e:admin` Playwright suite.

4.9 Observability & Third-Party Integrations

- **Sentry** — identical `@sentry/react` setup in both apps (10% trace sampling, error boundary wrapping the root component, disabled when no DSN is configured).
- **Razorpay** — Checkout.js loaded as a global script in `frontend/index.html`; `paymentService.ts` wraps it in a Promise-based helper (`openRazorpayModal`) plus an exponential-backoff saga-status poller (`pollSagaStatus`) for the async order→payment→entitlement flow.
- **Google Analytics (gtag.js)** — hard-coded measurement ID directly in `index.html` (not environment-driven, so dev/staging traffic reports into the same GA property as production — flagged in §4.11).
- **Anthropic Claude** (Admin Portal) — called *directly from the browser* via `@anthropic-ai/sdk` with `dangerouslyAllowBrowser: true`, using an API key read from `VITE_CLAUDE_API_KEY`. This is used exclusively to generate first-draft content for Levels 1, 2, 4, 5, 6, 7 (see §5.2) which an admin then edits before publishing.
- **Groq** — called directly from the browser (raw `fetch` to `api.groq.com`) from three internal team-tool pages only (WarRoom, GrowVioAgents), not from any user-facing or content-generation path.

4.10 Strengths

- **Single, disciplined API boundary.** 100% of backend traffic funnels through one axios instance per app with zero stray fetch/axios calls — makes cross-cutting changes (auth headers, retry, logging) a one-file change.
- **Aggressive, well-targeted code-splitting** in the User App — the auth-critical path stays in the main bundle while every heavy authenticated page is its own chunk.
- **Strong typed domain layer** — every service function has explicit request/response TypeScript types, which is what made this document's contracts derivable directly from source rather than guessed.
- **Real E2E coverage** across both apps via Playwright, including a dedicated API-coverage suite — uncommon rigor for a project of this size.
- **Thoughtful UX-for-infra details** — the 120s login timeout for cold starts, exponential-backoff payment polling, and a visible non-prod environment banner all show attention to real operational conditions rather than the happy path only.
- **Consistent, well-factored TanStack Query hooks** in the Admin Portal (`useBooks`, `useContent`) with correct cache-invalidation on every mutation.

4.11 Risks & Recommendations

Duplicate admin surface. The User App ships its own `/admin/*` route tree (`pages/admin/*.tsx`) alongside the dedicated Admin Portal app. Two of its five pages (`AdminUsers` , `AdminAnalytics`) are wired to real live endpoints (`/api/admin/users` , `/api/admin/analytics/active-users`); the other three (`AdminBooks` , `AdminOverview` , `AdminUpload`) render entirely hard-coded mock data with no API calls at all. This is dead/legacy weight that also doubles the attack surface for admin-role checks. **Recommendation:** retire the in-app admin pages now that the standalone Admin Portal exists, or explicitly document why both must remain.

JWT in `localStorage` . Both apps store the bearer token in `localStorage` (not an `httpOnly` cookie), which is readable by any script on the page — standard XSS exposure. Given the Admin Portal also loads a third-party LLM SDK directly in the browser with an API key in the same storage-accessible context, the blast radius of an XSS bug is higher than typical. **Recommendation:** consider `httpOnly` session cookies for the Admin Portal at minimum, and route Claude calls through a thin backend proxy instead of shipping the Anthropic key to the browser.

Inconsistent admin role gating. The User App's `AdminRoute` treats `ADMIN` , `SUPER_ADMIN` and `CONTENT_CREATOR` as equivalent (`isAnyAdmin`), while the Admin Portal's `ProtectedRoute` accepts only the literal role `'ADMIN'` and hard-blocks everyone else with no fallback UI. A `CONTENT_CREATOR` account — a role that clearly exists for exactly this app — cannot use the Admin Portal at all today.

`.env.dev` / `.env.staging` / `.env.preprod` are silently ignored by `npm run dev` . Vite only loads these non-standard filenames when invoked with a matching `--mode` flag; the plain dev script doesn't pass one, so a developer relying on `.env.dev` for local work is actually running on the code-level fallback (`http://localhost:8080`) or an empty value, not the file they edited. Confirmed directly in this session. **Recommendation:** either add `"dev:remote": "vite --mode dev"` style scripts, or rename to Vite's standard `.env.development` .

Stray root `package.json` . The repository root has its own `package.json` depending on `@anthropic-ai/sdk` , unrelated to either app's build (each has its own independent dependency tree). Untracked in git at the time of writing. Likely leftover from tooling setup — worth removing or documenting.

Two competing admin-book endpoints. The User App's own `bookService.create/update/remove` post directly to `/api/books` (the public, non-admin-gated gateway route) while the Admin Portal correctly uses `/api/admin/books` (gateway-enforced `AdminRoleFilter`). Any authenticated non-admin user's JWT can technically hit the first path; enforcement there depends entirely on content-service's own internal checks rather than the gateway. Worth auditing content-service to confirm it re-checks role on that path, or removing the unused capability from the frontend service file.

Environment-agnostic Google Analytics. The GA measurement ID is hard-coded in `index.html` rather than driven by `VITE_APP_ENV` , so dev/staging/preprod traffic pollutes production analytics. Low severity, easy fix.

5. Domain Model

5.1 Roles & Subscription Plans

Role	Meaning
USER	Standard reader — default role on signup
CONTENT_CREATOR	Can author/edit book content (Admin Portal target audience)
ADMIN	Full admin — books, users, analytics
SUPER_ADMIN	Highest privilege tier

Plan	Monthly Price	Unlocks
FREE	₹0	Levels 1–3 on featured-free books
STARTER	₹149	All books, all 7 levels (text), full gamification, quizzes/workbooks/notes
PREMIUM	₹299	Everything in Starter + audio/video summaries (Hindi & English) + priority support
PRO	₹799	Everything in Premium + AI RAG on 5 books/month + upload own PDFs + 2× XP multiplier

5.2 The 7-Level Knowledge Pyramid

Every book is progressively summarised into 7 levels of increasing depth. Admins generate a first AI draft per level (Claude, via the Admin Portal's `aiGenerationService.ts`), then hand-edit and publish it. Levels gate by required plan.

#	Name	Approx. Length	Gate	Content Shape
1	 Snapshot	150 words / 10 key points	FREE	<code>{ keyPoints: [{ order, heading, description, emoji, impactScore }] }</code>
2	 Flashdeck	500 words / 8–10 cards	FREE	<code>{ cards: [{ order, title, body, visualDirection, quote, cardType, colorMood, imageUrl? }] }</code>
3	 Infosummary	1200 words / 2-page visual	FREE	Image-led summary; admin uploads a single hero image via the level-3 upload endpoint
4	 Deep Read	3000 words / chapter summaries	STARTER	<code>{ pdfUrl, pdfKey, pageCount, inlineText (markdown), wordCount }</code>
5	 Mastery Quiz	~50 questions	PREMIUM	<code>{ totalQuestions, easyCount, mediumCount, hardCount, questions: [{ id, order, difficulty, question, options[4], correctAnswer, explanation, conceptTested }] }</code>

6	 Action Plan	8000 words / personal workbook	PREMIUM	<code>{ daily: WorkbookFormat, weekly: WorkbookFormat, monthly: WorkbookFormat }</code> where each has <code>{ title, sections: [{ title, prompt, placeholder }] }</code>
7	 Revision Booklet	15000 words / cheat sheet	PRO	<code>{ richText (markdown), wordCount }</code>

Each level also carries a lifecycle independent of the book itself: **DRAFT → PUBLISHED** , with versioning, rollback, scheduled publishing, an editor-lock (for collaborative editing) and A/B testing support — see §7.10.

5.3 Core Entities

Entity	Key Fields
User	<code>id, email, name, role, plan, emailVerified</code>
Book	<code>id, slug, title, hindiTitle, author, genre, coverImageUrl, isPremium, isFeaturedFree, rating, totalReads, tags[], keyPoints[], freeSummary, premiumSummary, eli5Summary, hindiFreeSummary, publishStatus</code>
Level	<code>level, type, title, status, requiredPlan, content, aiGeneratedAt, updatedAt</code>
Review	<code>id, bookId, userId, name, rating, text, helpfulCount, createdAt</code>
Note	<code>bookId, text, highlight</code>
Notification	<code>id, userId, type, title, message, read, createdAt</code>
UserStats (gamification)	<code>userId, totalPoints, weeklyPoints, streakDays, badges[], userLevel, nextLevelXP, weeklyXP, weeklyRank</code>

6. API Gateway Routing Map

All backend calls resolve to `VITE_GATEWAY_URL` / `VITE_API_URL` — the Spring Cloud Gateway at port 8080 in local dev. The gateway applies JWT/role filters and circuit breakers *before* proxying to the owning microservice, per its route config:

Path prefix	Routed to	Gateway filters
<code>/api/auth/**</code>	user-service	IP rate-limit (5 req/s, burst 10) · circuit breaker · no JWT
<code>/api/users/**</code>	user-service	JWT required · circuit breaker
<code>/api/payments/**</code>	user-service	JWT required · circuit breaker
<code>/api/books/**</code>	content-service	JWT required · circuit breaker (no admin-role filter — see §4.11 risk)
<code>/api/admin/books/**</code>	content-service	JWT + AdminRoleFilter · circuit breaker
<code>/api/admin/**</code> (everything else)	user-service	JWT + AdminRoleFilter · circuit breaker
<code>/ai/**</code>	content-service → Python AI sidecar	JWT required · circuit breaker
<code>/api/social/**</code>	social-service	JWT required (production profile)
<code>/api/gamification/**</code>	gamification-service	JWT required (production profile)
<code>/api/notifications/**</code>	notification-service	JWT required (production profile)

Note: `/api/admin/analytics/content-costs`, `/api/admin/users` and `/api/admin/analytics/active-users` all fall under the generic `/api/admin/**` catch-all and therefore route to **user-service**, not content-service — despite the first one being conceptually about content generation cost. Worth knowing when debugging or extending these.

7. Complete API Inventory

Every entry below reflects the exact request/response shape used in the TypeScript service layer at the time of writing. Unless noted, all responses are JSON with `Content-Type: application/json` ; all authenticated calls send `Authorization: Bearer <jwt>` (attached automatically by the axios interceptor).

7.1 Authentication & Session

POST

/api/auth/signup

Public

Create a new user account. Response is used to auto-login (same shape as Login).

Used by

frontend/src/services/authService.ts → `signup()` , consumed in pages/Signup.tsx

REQUEST BODY

```
{
  "name": "Riya Sharma",
  "email": "riya.sharma@gmail.com",
  "password": "Str0ngPass!23"
}
```

RESPONSE 200

```
{
  "id": 10432,
  "name": "Riya Sharma",
  "email": "riya.sharma@gmail.com",
  "role": "USER",
  "plan": "FREE",
  "emailVerified": false,
  "createdAt": "2026-08-11T06:12:03Z",
  "token": "eyJhbGciOiJIUzI1NiIs...
}
```

POST `/api/auth/login` Public

Authenticate with email/password. Shared by both apps. Client timeout is overridden to 120s (cold-start tolerance). Rate-limited by gateway: 5 req/s per IP, burst 10.

Used by	frontend authService.ts (full user object consumed) · admin authService.ts <code>loginAdmin()</code> (only <code>token/role/email</code> consumed)
----------------	--

REQUEST BODY

```
{
  "email": "riya.sharma@gmail.com",
  "password": "Str0ngPass!23"
}
```

RESPONSE 200

```
{
  "id": 10432,
  "name": "Riya Sharma",
  "email": "riya.sharma@gmail.com",
  "role": "USER",
  "plan": "STARTER",
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

RESPONSE 401 (INVALID CREDENTIALS)

```
{ "error": "INVALID_CREDENTIALS", "message": "Email or password is incorrect" }
```

POST `/api/auth/logout` JWT

Invalidate the current session server-side (client also clears localStorage independently).

REQUEST BODY

```
(none)
```

RESPONSE 200

```
{ "message": "Logged out" }
```

POST `/api/auth/resend-verification` Public

Resend the email-verification link.

REQUEST BODY

```
{ "email": "riya.sharma@gmail.com" }
```

RESPONSE 200

```
{ "message": "Verification email sent" }
```

GET /api/auth/verify-email?token=... Public

Consume the token from the verification email link.

QUERY PARAMS

```
?token=8f3c9e21-...-b6d4
```

RESPONSE 200

```
{ "message": "Email verified successfully"
}
```

POST /api/auth/forgot-password Public

Trigger a password-reset email.

REQUEST BODY

```
{ "email": "riya.sharma@gmail.com" }
```

RESPONSE 200

```
{ "message": "Password reset link sent if
the account exists" }
```

POST /api/auth/reset-password Public

Complete a password reset using the token from the email.

REQUEST BODY

```
{
  "token": "8f3c9e21-...-b6d4",
  "newPassword": "N3wStr0ngPass!"
}
```

RESPONSE 200

```
{ "message": "Password updated
successfully" }
```

7.2 User Profile, Notes & History

GET /api/users/me JWT

Fetch the current user — also used on app boot to validate a stored token.

REQUEST

```
(no body)
```

RESPONSE 200

```
{
  "id": 10432,
  "name": "Riya Sharma",
  "email": "riya.sharma@gmail.com",
  "role": "USER",
  "plan": "STARTER",
  "createdAt": "2026-01-14T09:02:11Z"
}
```

PUT /api/users/me **JWT**

Update the current user's display name.

REQUEST BODY

```
{ "name": "Riya S. Sharma" }
```

RESPONSE 200

```
{  
  "id": 10432, "name": "Riya S. Sharma",  
  "email": "riya.sharma@gmail.com",  
  "role": "USER", "plan": "STARTER"  
}
```

GET /api/users/history **JWT**

Reading history across all books the user has opened.

REQUEST

(no body)

RESPONSE 200

```
[  
  {  
    "bookId": "bk_9f21",  
    "bookTitle": "Atomic Habits",  
    "bookSlug": "atomic-habits",  
    "coverImageUrl":  
    "https://cdn.growv.io/covers/atomic-habits.jpg",  
    "progressPercent": 62,  
    "levelReached": 4,  
    "lastReadAt": "2026-08-09T18:44:00Z"  
  }  
]
```

POST /api/users/books/{bookId}/favorite **JWT**

Toggle a book as favorited for the current user.

REQUEST

(no body)
path: bookId = "bk_9f21"

RESPONSE 200

```
{ "bookId": "bk_9f21", "isFavorited": true  
}
```

POST `/api/users/books/{bookId}/shortlist` **JWT**

Toggle a book on the user's shortlist ("read next").

REQUEST

```
(no body)
path: bookId = "bk_9f21"
```

RESPONSE 200

```
{ "bookId": "bk_9f21", "isShortlisted":
  true }
```

GET `/api/users/notes?bookId={bookId}` **JWT**

List the current user's notes/highlights for a book.

QUERY PARAMS

```
?bookId=bk_9f21
```

RESPONSE 200

```
[
  {
    "id": "note_881",
    "bookId": "bk_9f21",
    "text": "Habit stacking > relying on
willpower",
    "highlight": "You do not rise to the
level of your goals...",
    "createdAt": "2026-08-09T18:50:00Z"
  }
]
```

POST `/api/users/notes` **JWT**

Create a note, optionally attached to a text highlight.

REQUEST BODY

```
{
  "bookId": "bk_9f21",
  "text": "Habit stacking > relying on
willpower",
  "highlight": "You do not rise to the
level of your goals..."
}
```

RESPONSE 201

```
{
  "id": "note_881", "bookId": "bk_9f21",
  "text": "Habit stacking > relying on
willpower",
  "createdAt": "2026-08-11T06:20:00Z"
}
```

DELETE /api/users/notes/{noteId} **JWT**

Delete a note.

REQUEST

```
(no body)
path: noteId = "note_881"
```

RESPONSE 204

```
(empty)
```

PUT /api/users/books/{bookId}/workbook **JWT**

Save the user's Level-6 (Action Plan) workbook answers, keyed by prompt id.

REQUEST BODY

```
{
  "answers": {
    "daily_1": "Read 10 pages every
morning at 7am",
    "weekly_1": "Reduced phone-checking
from 40/day to 15/day"
  }
}
```

RESPONSE 200

```
{ "bookId": "bk_9f21", "saved": true }
```

7.3 Books — Public / User

GET /api/books **JWT**

List/browse books. Accepts arbitrary filter params (genre, tag, page, size, etc. — passed through as-is).

QUERY PARAMS (EXAMPLE)

```
?genre=Self-Help&page=0&size=20
```

RESPONSE 200

```
[
  {
    "id": "bk_9f21", "slug": "atomic-
habits",
    "title": "Atomic Habits", "author":
"James Clear",
    "genre": "Self-Help",
    "coverImageUrl":
"https://cdn.growv.io/covers/atomic-
habits.jpg",
    "isPremium": false, "rating": 4.7,
    "totalReads": 18342,
    "tags": ["habits", "productivity"]
  }
]
```

GET `/api/books/{id}` **JWT**

Fetch one book by id.

REQUEST

```
path: id = "bk_9f21"
```

RESPONSE 200

```
{
  "id": "bk_9f21", "slug": "atomic-
  habits",
  "title": "Atomic Habits", "author":
  "James Clear",
  "genre": "Self-Help", "isPremium":
  false,
  "freeSummary": "Small habits compound
  into...",
  "keyPoints": ["Systems > goals", "..."],
  "rating": 4.7, "totalReads": 18342
}
```

GET `/api/books/slug/{slug}` **JWT**

Fetch one book by its URL slug (used for `/books/:bookSlug` route).

REQUEST

```
path: slug = "atomic-habits"
```

RESPONSE 200

```
{ "id": "bk_9f21", "slug": "atomic-
  habits", "title": "Atomic Habits", "...":
  "..." }
```

GET `/api/books/popular` **JWT**

Trending/most-read books for discovery surfaces.

REQUEST

```
(no params)
```

RESPONSE 200

```
[ { "id": "bk_9f21", "title": "Atomic
  Habits", "totalReads": 18342, "...": "..."
  } ]
```

GET `/api/books/search?q=...` **JWT**

Full-text book search.

QUERY PARAMS

```
?q=habits
```

RESPONSE 200

```
{
  "books": [ { "id": "bk_9f21", "title":
    "Atomic Habits", "...": "..." } ],
  "totalHits": 1,
  "backend": "elasticsearch",
  "query": "habits",
  "durationMs": 14
}
```

GET `/api/books/{bookId}/level/{level}` **JWT**

Fetch the published content of one level of one book, plan-gated server-side.

REQUEST

```
path: bookId = "bk_9f21", level = 1
```

RESPONSE 200

```
{
  "success": true,
  "data": {
    "level": 1,
    "content": {
      "keyPoints": [
        { "order": 1, "heading": "Your
        brain is lying to you every morning",
          "description": "...", "emoji":
        "🧠", "impactScore": "MINDBLOWN" }
      ]
    }
  }
}
```

RESPONSE 403 (PLAN TOO LOW)

```
{ "success": false, "error": "PLAN_REQUIRED", "requiredPlan": "STARTER" }
```

GET `/api/books/{bookId}/status` **JWT**

Per-book progress/status snapshot for the current user (levels unlocked, last position, etc).

REQUEST

```
path: bookId = "bk_9f21"
```

RESPONSE 200

```
{
  "bookId": "bk_9f21",
  "levelsUnlocked": [1, 2, 3, 4],
  "currentLevel": 4,
  "progressPercent": 62
}
```

POST `/api/books` **JWT** gateway does not enforce admin role here — see §4.11

Create a book (present in the User App's `bookService.ts` but not observed to be called from any current page — likely legacy/unused).

REQUEST BODY

```
{ "title": "Deep Work", "author": "Cal Newport", "genre": "Productivity" }
```

RESPONSE 201

```
{ "id": "bk_a112", "title": "Deep Work", "...": "..." }
```

PUT `/api/books/{id}` **JWT**

Update a book (same legacy-path caveat as above — the Admin Portal uses `/api/admin/books/{id}` instead).

REQUEST BODY

```
{ "title": "Deep Work (2nd ed.)" }
```

RESPONSE 200

```
{ "id": "bk_a112", "title": "Deep Work (2nd ed.)" }
```

DELETE `/api/books/{id}` **JWT**

Delete a book (same legacy-path caveat).

REQUEST

```
path: id = "bk_a112"
```

RESPONSE 204

```
(empty)
```

7.4 Payments

POST `/api/payments/create-order` **JWT**

Create a Razorpay order server-side for a plan upgrade. Response feeds directly into the Razorpay Checkout modal.

REQUEST BODY

```
{ "plan": "PREMIUM", "couponCode":  
  "LAUNCH20" }
```

RESPONSE 200

```
{  
  "razorpayOrderId": "order_NqZ8x...",  
  "keyId": "rzp_live...",  
  "amountPaise": 23920,  
  "currency": "INR",  
  "plan": "PREMIUM"  
}
```

POST `/api/payments/verify?plan={plan}` **JWT**

Verify the Razorpay signature after checkout completes and kick off the entitlement-grant saga.

REQUEST BODY

```
{  
  "razorpayOrderId": "order_NqZ8x...",  
  "razorpayPaymentId": "pay_NqZAb...",  
  "razorpaySignature": "9ef4d4c9a3b2..."  
}
```

RESPONSE 200

```
{  
  "sagaId": "saga_7712",  
  "status": "PROCESSING",  
  "completed": false  
}
```

GET `/api/payments/checkout/status/{sagaId}` **JWT**

Poll the async payment saga until entitlement is granted. Client polls with exponential backoff (1s → 8s cap, 12 attempts max).

REQUEST

```
path: sagaId = "saga_7712"
```

RESPONSE 200 (IN PROGRESS)

```
{ "sagaId": "saga_7712", "status":  
  "PROCESSING", "completed": false }
```

RESPONSE 200 (DONE)

```
{ "sagaId": "saga_7712", "status": "SUCCESS", "completed": true, "plan": "PREMIUM" }
```

7.5 Gamification

GET `/api/gamification/stats/{userId}` **JWT**

XP, streak, level and badges for a user (dashboard + profile).

REQUEST

```
path: userId = 10432
```

RESPONSE 200

```
{
  "userId": 10432,
  "totalPoints": 4820,
  "weeklyPoints": 340,
  "streakDays": 12,
  "userLevel": 6,
  "nextLevelXP": 5000,
  "weeklyXP": 340,
  "weeklyRank": 18,
  "badges": [
    { "name": "STREAK_7", "iconUrl":
      "https://cdn.growvio.in/badges/streak7.png",
      "unlockedAt": "2026-07-30T00:00:00Z"
    }
  ]
}
```

GET `/api/gamification/leaderboard?limit=&type=` **JWT**

Leaderboard entries. `type` ∈ `weekly | all-time` (service normalises `ALL_TIME` → `all-time`).

QUERY PARAMS

```
?limit=10&type=weekly
```

RESPONSE 200

```
[
  { "rank": 1, "name": "Arjun Mehta",
    "xp": 5210, "books": 14, "trend": "up" },
  { "rank": 2, "name": "Riya Sharma",
    "xp": 4820, "books": 11, "trend": "same" }
]
```

7.6 Social / Reviews

GET `/api/social/books/{bookId}/reviews?page=&size=` **JWT**

Paginated reviews for a book. Default `page=0, size=20`.

QUERY PARAMS

```
?page=0&size=20
```

RESPONSE 200

```
[
  {
    "id": "rev_331", "bookId": "bk_9f21",
    "userId": 10432,
    "name": "Riya Sharma", "rating": 5,
    "text": "Changed how I build habits at
work.",
    "helpfulCount": 12, "createdAt":
    "2026-08-01T10:00:00Z"
  }
]
```

POST `/api/social/books/{bookId}/reviews` **JWT**

Submit a review for a book.

REQUEST BODY

```
{ "rating": 5, "text": "Changed how I
build habits at work." }
```

RESPONSE 201

```
{
  "id": "rev_331", "bookId": "bk_9f21",
  "userId": 10432,
  "rating": 5, "text": "Changed how I
build habits at work.",
  "helpfulCount": 0, "createdAt": "2026-
08-11T06:30:00Z"
}
```

GET `/api/social/books/{bookId}/rating` **JWT**

Aggregate rating for a book.

REQUEST

```
path: bookId = "bk_9f21"
```

RESPONSE 200

```
{ "averageRating": 4.7, "totalReviews":
1820 }
```

POST `/api/social/reviews/{reviewId}/helpful` **JWT**

Mark a review as helpful (increments counter).

REQUEST

```
path: reviewId = "rev_331"
```

RESPONSE 200

```
{ "id": "rev_331", "helpfulCount": 13 }
```

7.7 Notifications

GET `/api/notifications/history/{userId}` **JWT**

Notification history for a user.

REQUEST

```
path: userId = 10432
```

RESPONSE 200

```
[
  {
    "id": "notif_5521", "userId": 10432,
    "type": "STREAK_REMINDER",
    "title": "Keep your streak alive! 🔥",
    "message": "You're 1 day from a 14-day streak.",
    "read": false, "createdAt": "2026-08-11T05:00:00Z"
  }
]
```

7.8 AI Generation (Backend)

Routed through `/ai/**` → content-service → Python AI sidecar. Distinct from the Admin Portal's *direct-to-Claude* browser calls in §7.10/§4.9 — this is the User App's own AI feature for turning a user-supplied PDF or pasted text into a summary.

POST `/ai/generate-summary` **JWT**

Upload a PDF for AI summarisation (PRO plan feature — "upload your own PDFs").

REQUEST

```
Content-Type: multipart/form-data
file:
```

RESPONSE 200

```
{
  "summary": "This document argues that...",
  "keyPoints": ["...", "..."],
  "wordCount": 812
}
```

POST `/ai/generate-from-text` **JWT**

Generate a summary from pasted text instead of a PDF.

REQUEST BODY

```
{
  "text": "Long pasted article or
notes...",
  "title": "My Notes",
  "author": "Unknown",
  "bookId": "bk_9f21"
}
```

RESPONSE 200

```
{ "summary": "...", "keyPoints": [...],
  "..."], "wordCount": 340 }
```

7.9 Admin — Books CRUD & Publish

All endpoints below require JWT + **ADMIN** role (gateway **AdminRoleFilter** on `/api/admin/books/**`).

GET `/api/admin/books?status=&page=&size=` **Admin**

Paginated book list for the admin table. `status` ∈ **ALL | DRAFT | PUBLISHED | ARCHIVED**.

QUERY PARAMS

```
?status=ALL&page=0&size=20
```

RESPONSE 200

```
{
  "books": [
    {
      "id": "bk_9f21", "title": "Atomic
Habits", "author": "James Clear",
      "genre": "Self-Help", "isPremium":
false,
      "publishStatus": "PUBLISHED",
      "levelsPublished": 7, "hasPdf":
true, "hasInfographic": true,
      "createdAt": "2026-01-02T00:00:00Z",
      "updatedAt": "2026-07-20T00:00:00Z"
    }
  ],
  "total": 128,
  "page": 0
}
```

GET `/api/admin/books/{bookId}` **Admin**

Full admin detail view of a book, including its 7 levels' metadata.

REQUEST

```
path: bookId = "bk_9f21"
```

RESPONSE 200

```
{
  "id": "bk_9f21", "title": "Atomic
Habits", "author": "James Clear",
  "genre": "Self-Help", "isPremium":
false, "rating": 4.7, "totalReads": 18342,
  "tags": ["habits", "productivity"],
  "publishStatus": "PUBLISHED",
  "referenceFiles": {
    "summaryPdfUrl":
"https://cdn.growvio.in/refs/atomic-
habits.pdf",
    "infographicUrl":
"https://cdn.growvio.in/refs/atomic-
habits-info.png"
  },
  "levels": [
    { "level": 1, "type": "SNAPSHOT",
"title": "Snapshot", "status":
"PUBLISHED",
      "requiredPlan": "FREE",
      "aiGeneratedAt": "2026-01-02T10:00:00Z",
      "updatedAt": "2026-01-03T09:00:00Z"
    }
  ],
  "createdAt": "2026-01-02T00:00:00Z",
  "updatedAt": "2026-07-20T00:00:00Z"
}
```

POST /api/admin/books Admin

Create a new book record (before any level content exists).

REQUEST BODY

```
{
  "title": "Deep Work",
  "hindiTitle": "गहन कार्य",
  "author": "Cal Newport",
  "genre": "Productivity",
  "tags": ["focus", "productivity"],
  "coverImageUrl":
    "https://cdn.growv.io/covers/deep-work.jpg",
  "isPremium": false,
  "description": "A guide to focused
    success in a distracted world."
}
```

RESPONSE 201

```
{
  "id": "bk_a112", "title": "Deep Work",
  "author": "Cal Newport",
  "publishStatus": "DRAFT", "levels": [],
  "createdAt": "2026-08-11T06:40:00Z",
  "updatedAt": "2026-08-11T06:40:00Z"
}
```

PUT /api/admin/books/{bookId} Admin

Update book metadata.

REQUEST BODY

```
{ "title": "Deep Work (Revised)", "tags":
  ["focus", "deep-work"] }
```

RESPONSE 200

```
{ "id": "bk_a112", "title": "Deep Work
  (Revised)", "...": "..." }
```

DELETE /api/admin/books/{bookId} Admin

Delete a book entirely.

REQUEST

path: bookId = "bk_a112"

RESPONSE 204

(empty)

POST /api/admin/books/{bookId}/publish Admin

Publish a book (makes it visible/discoverable to users).

REQUEST

(no body)

RESPONSE 200

```
{ "id": "bk_a112", "publishStatus":
  "PUBLISHED" }
```

POST `/api/admin/books/{bookId}/unpublish` Admin

Unpublish a book (hides it from discovery without deleting it).

REQUEST

(no body)

RESPONSE 200

```
{ "id": "bk_a112", "publishStatus":  
  "DRAFT" }
```

7.10 Admin — Content / Levels Editor

Powers the Tiptap-based rich content editor ([RichEditorPage](#)) and its AI-draft, versioning, scheduling, locking and A/B-testing subsystems.

GET `/api/admin/books/{bookId}/levels` Admin

List all 7 levels' metadata for a book.

REQUEST

path: bookId = "bk_9f21"

RESPONSE 200

```
[  
  { "level": 1, "type": "SNAPSHOT",  
    "status": "PUBLISHED", "requiredPlan":  
    "FREE" },  
  { "level": 2, "type": "FLASHDECK",  
    "status": "DRAFT", "requiredPlan": "FREE"  
  }  
]
```

GET `/api/admin/books/{bookId}/levels/{level}` Admin

Fetch one level's full content + metadata for editing.

REQUEST

path: bookId = "bk_9f21", level = 1

RESPONSE 200

```
{  
  "level": 1, "type": "SNAPSHOT",  
  "status": "DRAFT", "requiredPlan": "FREE",  
  "content": { "keyPoints": [ { "order":  
    1, "heading": "...", "description": "..."  
  } ] },  
  "aiGeneratedAt": "2026-08-11T06:00:00Z",  
  "updatedAt": "2026-08-11T06:00:00Z"  
}
```

PUT `/api/admin/books/{bookId}/levels/{level}` Admin

Overwrite a level's content/status/plan gate — the endpoint the AI-generation helpers (`aiGenerationService.ts`, §4.9) call after Claude returns a draft.

REQUEST BODY

```
{
  "content": { "keyPoints": [ { "order":
1, "heading": "...", "description": "...",
  "emoji": "🧠", "impactScore":
"MINDBLOWN" } ] },
  "status": "DRAFT",
  "requiredPlan": "FREE"
}
```

RESPONSE 200

```
{ "level": 1, "status": "DRAFT",
  "updatedAt": "2026-08-11T06:41:00Z" }
```

POST `/api/admin/books/{bookId}/levels/{level}/publish` Admin

Publish a single level.

REQUEST

(no body)

RESPONSE 200

```
{ "level": 1, "status": "PUBLISHED" }
```

POST `/api/admin/books/{bookId}/levels/{level}/unpublish` Admin

Revert a level to draft.

REQUEST

(no body)

RESPONSE 200

```
{ "level": 1, "status": "DRAFT" }
```

GET `/api/admin/books/{bookId}/levels/{level}/ai-draft` Admin

Fetch the raw AI-generated draft (pre-human-edit) for the rich editor's "AI draft" side panel.

REQUEST

path: bookId, level

RESPONSE 200

```
{
  "blocks": [ { "type": "heading", "text":
    "Your brain is lying to you every morning"
  },
    { "type": "paragraph",
    "text": "..." } ],
  "generatedAt": "2026-08-11T06:00:00Z"
}
```

PUT `/api/admin/books/{bookId}/levels/{level}/draft` Admin

Autosave the editor's in-progress block content (not yet published).

REQUEST BODY

```
{ "blocks": [ { "type": "heading", "text":
"..." } ] }
```

RESPONSE 200

```
{ "saved": true, "savedAt": "2026-08-
11T06:42:10Z" }
```

POST `/api/admin/books/{bookId}/levels/{level}/publish-final` Admin

Publish the human-edited final block content (distinct from the simpler `/publish` above, used specifically by the rich block editor flow).

REQUEST BODY

```
{ "blocks": [ { "type": "heading", "text":
"..." } ] }
```

RESPONSE 200

```
{ "level": 1, "status": "PUBLISHED",
"version": 3 }
```

GET `/api/admin/books/{bookId}/levels/{level}/published` Admin

Fetch the currently live (published) version, for comparison against the draft in the editor.

REQUEST

```
path: bookId, level
```

RESPONSE 200

```
{ "version": 2, "blocks": [ ... ],
"publishedAt": "2026-07-20T00:00:00Z" }
```

POST `/api/admin/books/{bookId}/levels/{level}/rollback` Admin

Roll a level back to a previous published version.

REQUEST BODY

```
{ "toVersion": 2 }
```

RESPONSE 200

```
{ "level": 1, "status": "PUBLISHED",
"version": 2 }
```

GET `/api/admin/books/{bookId}/levels/{level}/version-status` Admin

Diff/status between draft and published version (used to show "unpublished changes" indicators).

REQUEST

path: bookId, level

RESPONSE 200

```
{ "hasDraftChanges": true, "draftVersion":  
4, "publishedVersion": 3 }
```

GET `/api/admin/books/{bookId}/levels/{level}/preview` Admin

Rendered preview info (e.g. a signed preview URL) for the "preview as reader" feature.

REQUEST

path: bookId, level

RESPONSE 200

```
{ "previewUrl":  
"https://dev.growvio.in/preview/bk_9f21/1?  
token=...", "expiresAt": "2026-08-  
11T07:00:00Z" }
```

GET `/api/admin/books/{bookId}/supporting-docs` Admin

List uploaded reference materials (PDF, infographic, transcript) used as AI-generation context.

REQUEST

path: bookId

RESPONSE 200

```
[  
  { "type": "PDF", "url":  
    "https://cdn.growvio.in/refs/atomic-  
    habits.pdf", "uploadedAt": "2026-01-  
    02T09:00:00Z" }  
]
```

POST `/api/admin/books/{bookId}/levels/{level}/editor-lock` Admin

Acquire a collaborative-editing lock so two admins can't overwrite each other on the same level.

REQUEST

(no body)

RESPONSE 200

```
{
  "sessionId": "sess_ab12cd", "lockedBy":
  "priya@growvio.in",
  "expiresAt": "2026-08-11T06:47:00Z"
}
```

RESPONSE 409 (ALREADY LOCKED)

```
{ "error": "LOCKED", "lockedBy": "arjun@growvio.in", "expiresAt": "2026-08-11T06:45:00Z"
}
```

DELETE `/api/admin/books/{bookId}/levels/{level}/editor-lock` Admin

Release the editor lock on navigating away.

REQUEST BODY

```
{ "sessionId": "sess_ab12cd" }
```

RESPONSE 204

(empty)

PUT `/api/admin/books/{bookId}/levels/{level}/editor-lock/heartbeat` Admin

Keep an active lock alive while the admin is still editing.

REQUEST BODY

```
{ "sessionId": "sess_ab12cd" }
```

RESPONSE 200

```
{ "expiresAt": "2026-08-11T06:52:00Z" }
```

GET `/api/admin/books/{bookId}/editor-status` Admin

Lock status for every level of a book, keyed by level number — powers "🔒" being edited by Priya" badges in the book detail view.

REQUEST

path: bookId

RESPONSE 200

```
{
  "1": { "lockedBy": "priya@growvio.in",
  "expiresAt": "2026-08-11T06:47:00Z" }
}
```

POST `/api/admin/books/{bookId}/levels/{level}/schedule` Admin

Schedule a level to auto-publish at a future time.

REQUEST BODY

```
{ "scheduledAt": "2026-08-15T09:00:00Z" }
```

RESPONSE 200

```
{ "level": 1, "scheduledAt": "2026-08-15T09:00:00Z", "status": "DRAFT" }
```

DELETE `/api/admin/books/{bookId}/levels/{level}/schedule` Admin

Cancel a scheduled publish.

REQUEST

(no body)

RESPONSE 204

(empty)

POST `/api/admin/books/{bookId}/levels/{level}/ab-test/start` Admin

Start an A/B test between the current content (Variant A) and a new Variant B on this level.

REQUEST BODY

```
{ "variantBContent": "## Alternate framing of this level..." }
```

RESPONSE 200

```
{ "testId": "ab_5510", "level": 1, "status": "RUNNING", "startedAt": "2026-08-11T06:55:00Z" }
```

POST `/api/admin/books/{bookId}/levels/{level}/ab-test/conclude` Admin

End the test and declare a winning variant, which becomes the live content.

REQUEST BODY

```
{ "winner": "B" }
```

RESPONSE 200

```
{ "testId": "ab_5510", "level": 1, "status": "CONCLUDED", "winner": "B" }
```

7.11 Admin — File Uploads

All multipart/form-data with field name `file`.

POST `/api/admin/books/{bookId}/reference/pdf` Admin

Upload the reference summary PDF used as AI-generation context.

REQUEST

```
Content-Type: multipart/form-data
file:
```

RESPONSE 200

```
{
  "url":
    "https://cdn.growvio.in/refs/bk_9f21/summary.pdf",
  "key": "refs/bk_9f21/summary.pdf",
  "uploadedAt": "2026-08-11T06:58:00Z"
}
```

POST `/api/admin/books/{bookId}/reference/infographic` Admin

Upload a reference infographic image for AI context.

REQUEST

```
Content-Type: multipart/form-data
file:
```

RESPONSE 200

```
{ "url":
  "https://cdn.growvio.in/refs/bk_9f21/info.png",
  "key": "refs/bk_9f21/info.png" }
```

DELETE `/api/admin/books/{bookId}/reference/pdf` Admin

Remove the reference PDF.

REQUEST

(no body)

RESPONSE 204

(empty)

DELETE `/api/admin/books/{bookId}/reference/infographic` Admin

Remove the reference infographic.

REQUEST

(no body)

RESPONSE 204

(empty)

POST `/api/admin/books/{bookId}/levels/2/cards/{order}/image` Admin

Upload an image for one Level-2 (Flashdeck) card.

REQUEST

```
path: bookId, order = 3
Content-Type: multipart/form-data
file:
```

RESPONSE 200

```
{ "url":
  "https://cdn.growvio.in/levels/bk_9f21/2/c
  ard3.jpg" }
```

POST `/api/admin/books/{bookId}/levels/3/image` Admin

Upload the single hero image for Level 3 (Infosummary).

REQUEST

```
Content-Type: multipart/form-data
file:
```

RESPONSE 200

```
{ "url":
  "https://cdn.growvio.in/levels/bk_9f21/3/h
  ero.png" }
```

POST `/api/admin/books/{bookId}/levels/4/pdf` Admin

Upload the Level-4 (Deep Read) PDF; backend extracts text/page count for the reader.

REQUEST

```
Content-Type: multipart/form-data
file:
```

RESPONSE 200

```
{
  "pdfUrl":
    "https://cdn.growvio.in/levels/bk_9f21/4/d
    eep-read.pdf",
  "pdfKey": "levels/bk_9f21/4/deep-
    read.pdf",
  "pageCount": 22,
  "wordCount": 3120,
  "inlineText": "# Atomic Habits – Deep
    Read\n\n..."
}
```

7.12 Admin — Users & Analytics

These route to **user-service** via the generic `/api/admin/**` catch-all (see §6 callout). Called from both the standalone Admin Portal's future needs and, today, from the User App's in-app `/admin/*` pages (§4.11).

GET `/api/admin/users` Admin

List all platform users for the admin user-management table.

REQUEST

(no params observed in current usage)

RESPONSE 200

```
[
  { "id": 10432, "name": "Riya Sharma",
    "email": "riya.sharma@gmail.com",
    "role": "USER", "plan": "STARTER",
    "joinedAt": "2026-01-14T09:02:11Z" }
]
```

GET `/api/admin/analytics/active-users` Admin

Active-user analytics for the admin dashboard.

REQUEST

(no params)

RESPONSE 200

```
{
  "dau": 3420, "wau": 12800, "mau": 41200,
  "byPlan": { "FREE": 28000, "STARTER":
    9800, "PREMIUM": 2900, "PRO": 500 }
}
```

GET `/api/admin/analytics/content-costs` Admin

Tracks Claude/AI generation spend per book/level for the Admin Portal's Content Costs page.

REQUEST

(no params)

RESPONSE 200

```
{
  "totalSpendUsd": 184.32,
  "byBook": [
    { "bookId": "bk_9f21", "title":
      "Atomic Habits", "totalTokens": 82000,
      "costUsd": 2.46 }
  ]
}
```

GET `/api/admin/books/analytics` Admin

Book-level engagement analytics (reads, completion, ratings) — note this one *does* fall under content-service via the `/api/admin/books/**` prefix, unlike the two above.

REQUEST

(no params)

RESPONSE 200

```
{
  "topBooks": [ { "bookId": "bk_9f21",
    "title": "Atomic Habits", "totalReads":
    18342, "avgRating": 4.7 } ]
}
```

8. Environment Variables Reference

User App (frontend/)

Variable	Purpose	Example
VITE_GATEWAY_URL	API Gateway base URL	https://api-dev.growvio.in
VITE_APP_ENV	Drives isDev/isStaging/isPreprod/isProduction flags & EnvBanner	dev
VITE_APP_VERSION	Build version string (defaults to \$npm_package_version)	1.0.0
VITE_RAZORPAY_KEY_ID	Razorpay publishable key	rzp_test_placeholder
VITE_ENABLE_ANALYTICS	Gate for analytics side-effects	true / false
VITE_LOG_LEVEL	Client logger verbosity	debug
VITE_SENTRY_DSN	Sentry project DSN (empty disables Sentry)	—
VITE_PROD_TIER	Internal tiering flag	0
VITE_GROQ_API_KEY	Groq key for internal tool pages only	—

Admin Portal (admin/)

Variable	Purpose	Example
VITE_API_URL	API Gateway base URL — required , app throws at boot if unset	http://localhost:8080
VITE_CLAUDE_API_KEY	Anthropic key, used directly from the browser for AI draft generation	—
VITE_SENTRY_DSN	Sentry project DSN	—
VITE_APP_ENV	Sentry environment tag	dev

Local-dev override convention (documented in the repo [README.md](#) and confirmed working in this session): `frontend/.env.local` → `VITE_GATEWAY_URL` , `admin/.env.development.local` → `VITE_API_URL` . Both are gitignored and take precedence over the committed `.env.development` defaults.

9. Route Map Appendix

User App Routes

Path	Guard	Page
/	Public	Index (or ComingSoon in production)
/pricing, /plans	Public	PlansPage
/books	Public	BookDiscovery
/books/:bookSlug	Public	BookDetail
/search	Public	SearchResults
/author/:name, /category/:slug	Public	AuthorPage / CategoryPage
/signup, /login, /verify, /verify-email, /forgot-password, /reset-password	Public	Auth flow pages
/onboarding/interests, /goal, /first-book	Protected	Onboarding wizard
/home	Protected	Dashboard
/books/:id/level/:levelNum	Protected	BookReading
/books/:id/pyramid, /quiz, /notes, /workbook, /reviews, /share	Protected	KnowledgePyramidPage, Quiz, NotesHighlights, Workbook, BookReviews, ShareReferral
/profile, /profile/:userId, /profile/history, /profile/badges	Protected	UserProfile, ReadingHistory, BadgeCollection
/leaderboard, /community	Protected	Leaderboard, Community
/checkout, /payment/success	Protected	Checkout, PaymentSuccess
/admin, /admin/books, /admin/books/upload, /admin/books/:id/edit, /admin/users, /admin/analytics	AdminRoute (any admin role)	AdminOverview, AdminBooks, AdminUpload, LevelEditor, AdminUsers, AdminAnalytics — see §4.11 duplication note
/settings/profile, /notifications, /subscription, /goals	Protected	Settings pages
/team, /war-room, /task-tracker	AdminRoute	Internal tools (GrowVioAgents, WarRoom, TaskTracker)
/terms, /privacy	Public	TermsPage, PrivacyPage

Admin Portal Routes

Path	Guard	Page
/login	Public	LoginPage

/	Protected (ADMIN only)	DashboardPage
/books, /books/new, /books/:bookId, /books/:bookId/edit	Protected	BooksListPage, BookCreatePage, BookDetailPage, BookEditPage
/books/:bookId/content	Protected	ContentEditorPage
/books/:bookId/levels/:levelNumber/editor	Protected	RichEditorPage
/books/:bookId/levels/:levelNumber/ab-test	Protected	AbTestPage
/content-costs	Protected	ContentCostsPage